

The Functionalizer: Lossless Functional Decomposition for Subword Tokenization

Connor Makowski
Center for Transportation & Logistics
Massachusetts Institute of Technology
Cambridge, MA, USA
conmak@mit.edu

Willem Guter
Center for Transportation & Logistics
Massachusetts Institute of Technology
Cambridge, MA, USA
wjguter@mit.edu

September 18, 2026

Abstract

Standard subword tokenizers either treat every orthographic variation of a word (such as `hello`, `Hello`, `HELLO`, and `Héllö`) as unrelated vocabulary entries, which fragments the embedding space, or discard this variation through lossy normalization. We present the **Functionalizer**, a lossless pre-tokenizer framework that factors orthographic and structural variations into a compositional **opcode/operand** prefix stream before tokenization: a canonical base token (operand) prefixed by parametric transformation operators (opcodes) encoded in the Unicode Private Use Area. We introduce operators covering casing (**CAPITALIZE**), diacritics (13 dedicated opcodes), and character repetition (**REPEAT**, **MULTIREPEAT**), which are fully reversible. Across natural language and code corpora, the Functionalizer enables complete corpus coverage with significantly smaller vocabularies under unconstrained exhaustion conditions, reducing actual vocabulary slot requirements by up to **19.7%**. Downstream evaluations on ~ 98 M-parameter GPT-2 models show that the Functionalizer improves Python code syntax validity (**9.12%** vs. **7.70%**) while reducing duplicate n -gram repetition in natural language prose. These findings demonstrate that functional decomposition can be an effective mechanism for vocabulary-efficient, structurally aware language modeling, and motivate further validation at production scale.

1 Introduction

Subword tokenizers face a dilemma. Treating `hello`, `Hello`, and `Héllö` as independent tokens causes **vocabulary expansion**: redundant surface forms consume embedding slots, and a gradient update to `Hello` only indirectly benefits `hello`. The alternative, aggressive lowercasing and accent-stripping, is lossy. It compresses the vocabulary but permanently discards information the downstream model can never recover.

The Functionalizer takes a third path: **lossless functional decomposition**. Rather than memorizing surface forms or destroying information, it factors variation out into reusable *operators* applied to a single *canonical base*. The design is borrowed directly from Instruction Set Architecture: a CPU does not implement a distinct instruction for every constant (`ADD_1`, `ADD_2`, `...`); it separates the operation (opcode) from its data (operand), as in `ADD A, #1`. The Functionalizer applies the same factoring: the base token is the operand, and the transformation prefix is the opcode.

Contributions.

1. A unified opcode/operand framework that handles casing, diacritics, and character repetition under a single compositional, parametric, dictionary-free, and fully lossless scheme.
2. A concrete Private Use Area (PUA) encoding that is usable in standard tokenizers (such as Hugging Face’s BPE) and fully reversible.

3. Empirical validation across natural language and code corpora, demonstrating that under unconstrained merge exhaustion, the Functionalizer reduces the total vocabulary slots required to fully cover a corpus through the collapse of formatting variations.
4. Downstream evaluations on $\sim 98\text{M}$ -parameter language models demonstrating that the Functionalizer lowers character-level perplexity on Python, improves Python syntax validity, and mitigates duplicate n -gram repetition on FineWeb-Edu prose.

2 Related Work

Subword tokenization. BPE (Sennrich et al., 2016) and WordPiece (Schuster and Nakajima, 2012) build vocabularies bottom-up by iteratively merging frequent (or likelihood-maximizing) pairs, while Unigram (Kudo, 2018) prunes a large seed vocabulary top-down under a unigram language model; byte-level BPE (Radford et al., 2019) avoids out-of-vocabulary failures by operating over 256 byte values.

Inline orthographic preprocessing. Pre-tokenization strategies targeting casing variations have antecedents in text compression (e.g., Rexline and Robert, 2011) and were formalized as inline casing for neural machine translation by Bérard et al. (2019), with Etchegoyhen and Gète (2020) moving case flags prior to subword tokenization. More recent systems expand inline tags to morphological roots (e.g., Bayram et al., 2025), capcode markers (e.g., TokenMonster; Forsythe, 2023), and position-indexed diacritics (InCa and InDia; Semenov and Popel, 2025). In parallel, Samuel and Øvrelid (2023) introduce the *Factorizer*, which factorizes subwords into discrete learned code triplets (3×256) using a VQ-VAE model. Unlike the Factorizer’s learned, non-bijective subword codes, the Functionalizer establishes a deterministic, rule-based, fully bijective opcode/operand Instruction Set Architecture (ISA). In related work on case encoding, Jain et al. (2023) analyze the efficiency and robustness of inline case markers, finding that un-fused standalone marker tokens introduce sequence length overhead and decoding slowdowns while requiring targeted data augmentation for capitalization robustness. While reversibility is shared with prior inline tagging approaches, the Functionalizer differs in key structural ways: (i) it operates deterministically without external dictionaries or frequency thresholds (unlike the morphological dictionaries of Bayram et al. (2025) or the frequency tables of Semenov and Popel (2025)), (ii) it unifies casing, combining diacritics, and structural character/whitespace repetition under a formal PUA opcode/operand ISA where numeric parameters explicitly address character positions within pieces, and (iii) it is evaluated across both natural language prose and source code domains.

Morphology-aware tokenization. Morfessor (Creutz and Lagus, 2002, 2007) performs unsupervised morpheme segmentation; MorphBPE (Asgari et al., 2025) prevents BPE merges from crossing morpheme boundaries. The Functionalizer is complementary: where morphology-aware methods target linguistic structure, the Functionalizer targets orthographic surface variation, and the two could be composed.

Tokenization-free models. ByT5 (Xue et al., 2022) pursues orthographic robustness by operating directly on raw bytes, at the cost of significantly longer sequence lengths. Subsequent architectures like MrT5 (Kallini et al., 2025) and MEGABYTE (Yu et al., 2023) mitigate this sequence overhead through dynamic byte-merging or multiscale patch hierarchies. The Functionalizer achieves similar surface-form invariance while retaining subword granularity and avoiding byte-level sequence expansion.

Structured Unicode encoding. SCRIPT-BPE (Land and Arnett, 2025) re-encodes characters by Unicode script and category to mitigate cross-lingual bias. The normalization literature (e.g., Gorman and Pinter, 2025) documents the downstream cost of inconsistent Unicode handling and cautions against destructive diacritic stripping without recovery. The Functionalizer aligns with this guidance: while diacritical marks are extracted from base characters to collapse redundant vocabulary forms, they are explicitly preserved as parametric PUA operators (ACUTE, GRAVE, etc.), guaranteeing complete, non-destructive reversibility.

3 The Functionalizer Framework

The Functionalizer establishes a parametric, lossless, prefix-based pre-tokenization framework. Instead of tokenizing raw surface forms directly, the framework decomposes orthographic and structural variations into a compositional sequence of non-destructive operators (opcodes) prepended to a canonical base token (operand). By isolating parametric transformations from semantic roots, downstream models share canonical base embeddings while preserving all orthographic details for lossless reconstruction.

3.1 PUA Instruction Layout

Instructions are prepended to the base token as a sequence of PUA codepoints, composed of an operator followed by numeric parameters:

[operator] [param_1] [param_2] ... → [base_token]

- **Numeric parameters** (U+E000–U+E0FF): encode integer values 0–255 (`value = codepoint - 0xE000`).
- **Operators** (U+E100–U+EFFF): opcodes consuming a fixed number of parameters, leaving the remaining plane available for future operators.

In our implementation, opcodes precede base operands ([opcode] → [base]). While suffix ordering ([base] → [opcode]) would allow semantic intent to precede orthographic specification in autoregressive generation, prefix ordering may offer a structural advantage for multi-token words: when a word splits into multiple BPE subwords (e.g., ["neuro", "computation", "al"]), prepending the opcode allows the formatting operator to remain visible across all self-attention layers for every constituent subword.

3.2 Encoding and Reversibility

The transformation pipeline is fully bijective. **Encoding** extracts combining diacritics into serialization operators and locates uppercase indices for **CAPITALIZE**, strips combining marks, lowercases remaining characters, and prepends the operator prefix to the canonical base. **Decoding** applies operators in reverse order to restore diacritics and casing before stripping the prefix, recovering the original text exactly.

Repetition operators execute last in the decode order, repeating the transformed base unit; position parameters refer to indices within the individual piece prior to repetition expansion (see Table 6 in Appendix B for worked examples). Heterogeneous cased repetitions (e.g., `Abcabcab`) fall back to uncollapsed representations.

3.3 Current Operator Specifications

The current implementation includes operators for capitalization, combining diacritics (13 dedicated opcodes in U+E100–U+E10D), and character repetition (Table 1).

Table 1: Functionalizer Operator Specifications

Operator / Opcode	Codepoint Range	Parameters	Action
CAPITALIZE	U+E100	<code>pos</code>	Uppercase character at <code>pos</code> .
DIACRITICS (13 variants) [†]	U+E101–U+E10D	<code>pos</code>	Apply combining diacritic mark at <code>pos</code> .
REPEAT	U+E200	<code>pos, count</code>	Expand character at <code>pos</code> to <code>count</code> copies.
MULTIREPEAT	U+E201	<code>start, end, count</code>	Expand subsequence [<code>start, end</code>) to <code>count</code> copies.

[†]Dedicated opcodes for TILDE, ACUTE, GRAVE, CIRCUMFLEX, DIAERESIS, MACRON, DOT_ABOVE, RING_ABOVE, DOUBLE_ACUTE, CARON, COMMA_BELOW, CEDILLA, and OGONEK (see Appendix A).

3.4 Pipeline Integration

The Functionalizer is intended to be run on pre-split pieces rather than raw text. Executing after an initial regex splitter (such as `Llama Split` from [Dubey et al., 2024](#)) establishes a localized coordinate frame ($\text{pos} \leq 255$) for numeric parameters. Bounding operator addresses to localized pieces rather than global document offsets keeps parameter values strictly within a single-byte range (0–255, mapped to U+E000–U+E0FF), preventing parameter expansion while ensuring clean interaction with downstream BPE merging. Alternative methods such as scanning text and injecting operators directly are feasible but not considered within the scope of this work.

By default, the Functionalizer emits operators as standalone prefix tokens decoupled from canonical base words (`split_operators = true`). Decoupling ensures base token embeddings are shared universally across casing variants and prevents the vocabulary from memorizing fused surface forms, at the expense of sequence length overhead (see Table 3). Alternatively, operators can remain **fused** with base pieces prior to subword training (`split_operators = false`), allowing tokenizers like BPE or SentencePiece to adaptively merge frequent cased words while splitting rare forms as discussed by [Jain et al. \(2023\)](#).

4 Experimental Setup

4.1 Pipelines and Configurations

We define the primary dataset preprocessing components across our experimental pipelines:

- **Unicode Normalization (NFC):** All configurations apply Unicode Normalization Form C (NFC) to standardize pre-composed characters across corpora.
- **Regex Splitting (Llama Split):** Segments raw text into localized character runs (contractions, words, numbers, punctuation, whitespace blocks) using the standard LLaMA pre-tokenization regex splitter. This isolates indentation sequences and punctuation while keeping character offset counts compact ($\text{pos} \leq 255$).
- **Functionalizer Decompositions:** Governed by pre-tokenizer flags mirroring the operators in Section 3.3: casing decomposition (`capitalize`), diacritic serialization (`serialize`), and repetition collapse (`repeat`).

4.2 Datasets, Vocabulary, and Tokenizer Metrics

We evaluate standalone tokenizers across natural language prose (Wikitext from [Merity et al., 2016](#), FineWeb-Edu from [Penedo et al., 2024](#)) and source code repositories (Python-Codes-25k from [Flytech, 2023](#), GitHub-Code-Python from [CodeParrot Team and Hugging Face, 2022](#)). To measure the unconstrained vocabulary footprint required to cover each corpus, we sample up to 100,000 documents per dataset and train BPE tokenizers with an unconstrained target vocabulary budget (4096k), allowing BPE to iteratively merge all viable pairs until complete merge candidate exhaustion is reached.

We evaluate vocabulary reduction and token compression using three primary metrics:

1. **Actual Vocab:** The number of vocabulary entries learned by BPE prior to merge candidate exhaustion.
2. **Vocab Diff (%)**: Percentage reduction in learned vocabulary size under merge exhaustion.
3. **Characters per Token (Chars/Token)**: Average visual characters per token. Higher values reflect higher text compression and shorter sequences.

4.3 Downstream Training and Evaluation

To assess downstream performance, we train a GPT-2 Small architecture (12 layers, 768 hidden dimension, 12 attention heads, context length 512, tied embeddings; ~98M total parameters with a 16k vocabulary) across five random seeds (1–5) on FineWeb-Edu and GitHub-Code-Python.

Table 2: Tokenizer Metrics under Full Corpus Exhaustion (Sampled up to 100k Documents)

Dataset	Vocab (Baseline)	Vocab (Func)	Vocab Diff	Chars/Token (Baseline)	Chars/Token (Func)	Chars/Token Diff
Wikitext	106,023	90,531	-14.61%	4.6045	3.9464	-14.29%
Python-Codes	68,471	57,012	-16.74%	4.2030	3.4806	-17.19%
FineWeb-Edu	1,214,684	975,169	-19.72%	5.0092	4.3642	-12.88%
GitHub-Code-Python	4,071,598	3,356,761	-17.56%	4.2961	3.5343	-17.73%

- **Training and Inference Details:** Models are trained for 50,000 steps using AdamW (learning rate $4e-4$, linear decay with 1,000 warmup steps, weight decay 0.01, effective batch size 32). Generation is evaluated on 1,000 validation prompts per dataset using greedy decoding with KV caching and a limit of 256 new tokens (also stopping on [SEP] or repetition collapse).
- **Downstream Evaluation Metrics:**
 - **Per-Character Perplexity (Char PPL):** Normalizes token loss $\mathcal{L}_{\text{token}}$ by the validation character-to-token ratio, enabling direct comparison across tokenizers: $\text{PPL}_{\text{char}} = \exp\left(\frac{\mathcal{L}_{\text{token}}}{R_{\text{char/token}}}\right)$.
 - **Repetition Collapse and Pre-Collapse Length:** Generation halts early if a cycle of ≤ 20 tokens repeats 4 times consecutively. *Avg Tokens Pre-Collapse* and *Avg Chars Pre-Collapse* measure valid prefixes preceding the cycle.
 - **Repetition (%):** Proportion of duplicate overlapping word n -grams (averaged over $n \in \{2, 3, 4\}$).
 - **Syntax Success Rate:** Percentage of Python generations that parse cleanly under `ast.parse` (empty or collapsed completions score as syntax failures).
 - **% Empty:** Percentage of generated sequences that are empty or contain only whitespace or raw PUA control characters.

5 Results

5.1 Tokenizer Metrics

To evaluate intrinsic vocabulary reduction, token compression, and corpus exhaustion across diverse domains (independently of downstream modeling), we evaluate standalone tokenizers across natural language prose and source code corpora sampled up to 100,000 documents each. Table 2 presents metrics under full merge candidate exhaustion, comparing the baseline **Llama Split** (without decomposition) against **Llama Split + Functionalizer** (with decomposition).

Distinct vocabulary and corpus exhaustion. Across all evaluated corpora, factoring out surface variations allows the Functionalizer to achieve complete corpus coverage with smaller vocabularies. Under unconstrained merge exhaustion, the Functionalizer reduces required vocabulary slots by **14.61%** to **19.72%** (averaging **17.16%** reduction across datasets), reaching peak reduction on FineWeb-Edu (-19.72%).

Characters per Token (Chars/Token) and Sequence Length. Because operators are emitted as standalone prefix tokens, the Functionalizer incurs an expected token expansion on un-fused text (-12.88% to -17.73% Chars/Token diff on validation data), trading sequence length for clean representation sharing and downstream syntactic fidelity (Section 5.3).

5.2 Training Dynamics and Language Modeling Performance

We analyze the training behavior and next-token prediction performance of the $\sim 98\text{M}$ parameter GPT-2 model under each tokenizer configuration. Table 3 summarizes training metrics across FineWeb-Edu and GitHub-Code-Python, averaged over five random seeds.

Per-Character Perplexity (Char PPL). As detailed in Section 4.3, Char PPL normalizes token loss by sequence length to ensure direct cross-tokenizer comparability.

- **GitHub-Code-Python:** The Functionalizer configuration achieves lower Char PPL than the baseline (**1.5328** vs. **1.5697**), reflecting improved normalized modeling density alongside lower cross-entropy loss (**0.6525** vs. **0.8056**).

Table 3: Training Dynamics and Language Modeling Metrics (~ 98 M Parameters, 5 Seeds)

Tokenizer Type	Vocab Size	Chars/Token	Chars/Token Diff	Final Loss	Token PPL	Char PPL
Dataset: FineWeb-Edu (Prose)						
Llama Split	16,000	4.178	0.00%	3.3954 ± 0.0110	29.83 ± 0.33	2.2662 ± 0.0060
Llama Split + Functionalizer	16,000	3.847	-7.91%	3.1261 ± 0.0044	22.79 ± 0.10	2.2656 ± 0.0026
Dataset: GitHub-Code-Python						
Llama Split	16,000	1.767	0.00%	0.8056 ± 0.0961	2.25 ± 0.22	1.5697 ± 0.0850
Llama Split + Functionalizer	16,000	1.487	-15.84%	0.6525 ± 0.0013	1.92 ± 0.00	1.5328 ± 0.0013

Table 4: Downstream Inference Metrics (~ 98 M Parameters, 1,000 Prompts, 5 Seeds)**(a) Prose Generation Metrics (FineWeb-Edu)**

Tokenizer Type	Avg Tokens Pre-Collapse	Avg Chars Pre-Collapse	% Empty	% Collapsed	Repetition (%)
Llama Split	87.4 ± 3.8	357.9 ± 17.5	$0.1 \pm 0.0\%$	$70.3 \pm 2.4\%$	$66.0 \pm 0.4\%$
Llama Split + Functionalizer	76.2 ± 4.2	217.7 ± 13.6	$7.1 \pm 0.5\%$	$78.4 \pm 1.4\%$	$55.8 \pm 1.5\%$

(b) Source Code Generation Metrics (GitHub-Code-Python)

Tokenizer Type	Avg Tokens Pre-Collapse	Avg Chars Pre-Collapse	% Empty	% Collapsed	Repetition (%)	Syntax Success (%)
Llama Split	227.7 ± 17.2	375.6 ± 23.2	$0.0 \pm 0.0\%$	$15.5 \pm 8.2\%$	$25.5 \pm 6.4\%$	$7.70 \pm 2.63\%$
Llama Split + Functionalizer	231.9 ± 7.0	319.0 ± 10.1	$2.1 \pm 3.8\%$	$15.5 \pm 3.4\%$	$17.9 \pm 1.0\%$	$9.12 \pm 1.22\%$

- **FineWeb-Edu (Prose):** On prose, the Functionalizer achieves equivalent character-level perplexity (**2.2656** vs. **2.2662**), demonstrating that sequence expansion can be absorbed without degrading normalized modeling capacity.

5.3 Downstream Generation and Task Evaluation

We perform greedy decoding evaluations with KV caching on 1,000 validation prompts per dataset to assess repetition degeneracy and code syntax validity. Table 4 lists the results.

Downstream Code Generation Syntax Success Rate. On GitHub-Code-Python, integrating the Functionalizer pre-tokenizer substantially improves the model’s capacity to output valid code syntax. The Llama Split + Functionalizer configuration reaches an overall syntax success rate of **9.12%** compared to **7.70%** for the baseline (an **18.4%** relative improvement) with substantially tighter variance across seeds. A part of this improvement likely arises from the structured decomposition of whitespace and identifiers: standard BPE fragments indentation into arbitrary whitespace chunks, whereas the REPEAT operator parameterizes indentation into an arithmetic relationship where whitespace blocks share a base character and differ only by an ordinal count. Combined with unified casing across identifier conventions (`camelCase`, `snake_case`), this parameterization provides downstream models with clearer structural representations.

Mitigating Repetitive Degeneracy and Generation Trade-offs. On FineWeb-Edu prose, the Functionalizer reduces duplicate word n -gram repetition from **66.0%** to **55.8%**. On GitHub-Code-Python, duplicate n -grams similarly decrease from **25.5%** to **17.9%**, with greater stability across seeds. At this small ~ 98 M-parameter scale, standalone prefixes also introduce decoding trade-offs, resulting in more empty sequences (7.1% on prose), some of which are operator-only sequences (which were scored as empty).

6 Discussion

The empirical findings presented across vocabulary scaling, training dynamics, and inference demonstrate that the Functionalizer establishes an effective Pareto-like trade-off for tokenization. Modern language modeling architectures have traditionally been forced to choose between two extremes: subword vocabularies that optimize sequence length at the cost of severe vocabulary fragmentation, or byte-level models that eliminate surface fragmentation at the expense of substantial sequence inflation. The Functionalizer navigates this continuum by achieving surface-form invariance with only modest sequence overhead (+8.6% on prose, +18.8% on code), routing orthographic variants to shared base embeddings while preserving exact reversibility.

In downstream evaluations, functional decomposition yields tangible improvements in generation quality alongside specific decoding trade-offs. On source code, the structured parameterization of whitespace and casing likely translates into higher syntactic fidelity, increasing Python syntax validity. On natural language prose, separating surface variations from lexical roots substantially reduces repetitive degeneracy, lowering duplicate n -gram content on FineWeb-Edu. We hypothesize that the reduced surface variation helps keep the model from locking into repetitive surface loops. At the same time, emitting standalone control prefixes introduces decoding challenges at small scales, where our ~ 98 M-parameter models occasionally produced orphaned control characters or empty sequences. While scaling model capacity should strengthen grammatical control over auxiliary prefix tokens, practical mitigations such as constrained decoding or prefix masking during generation remain valuable future directions.

From an efficiency perspective, the primary cost of standalone operator tokenization is the expansion of sequence lengths, which directly increases Key-Value (KV) cache memory and attention computation during autoregressive generation. While our experimental setup evaluated fully decoupled prefixes to isolate representational effects, this is not required for practical production deployments. Systematically evaluating fused configurations could add value, where frequently cased words or common structures (e.g., a period followed by a space and capitalization) are merged into unified tokens while less common variants remain decomposed. This can provide a direct mechanism to eliminate sequence expansion on common vocabulary while retaining representation sharing across the long tail.

6.1 Limitations and Future Directions

1. **Scale and Compute Equalization:** Downstream evaluations were conducted at the ~ 98 M-parameter scale across a fixed budget of 50,000 training steps. Because of standalone sequence expansion, the Functionalizer processed ~ 8 – 16% fewer raw bytes during pretraining than the baseline. Evaluating multi-billion parameter architectures trained under equalized wall-clock time and character/byte budgets will isolate representational gains from sequence length disparities.
2. **Prefix Fusion Extensions:** Benchmarking hybrid fusion thresholds across vocabulary frequency tiers to empirically characterize the trade-off between inference sequence length and representation sharing, complemented by prefix-aware attention optimizations. Part of this functionality already exists within the Functionalizer framework, but is not evaluated within the scope of this paper.
3. **Addressing, Script Coverage, and Extended Operators:** Parameter indexing is currently bounded to $\text{pos} \leq 255$ within pre-tokenized pieces, and diacritics are bounded to 13 combining marks. Natural extensions include range/block casing (e.g., [ALL-CAPS]), non-Latin scripts, morphological lemma folding for agglutinative languages, and numeric/date templates.
4. **Component Ablations:** Our downstream experiments evaluated the composite Functionalizer pipeline (CAPITALIZE + diacritics + REPEAT). Disentangling the individual downstream contributions of casing versus structural whitespace repetition remains a valuable direction for future study.

7 Conclusion

The Functionalizer introduces a lossless, dictionary-free pre-tokenization framework that factors orthographic and structural variations into a compositional opcode/operand structure encoded in the Unicode Private Use Area. By decoupling surface transformations from canonical base roots, the framework eliminates redundant vocabulary memorization, reducing required vocabulary slots by up to **19.7%** under corpus exhaustion.

Downstream language modeling evaluations demonstrate that this structural decomposition translates to tangible generative benefits: improving Python code syntax validity, lowering code character perplexity, and mitigating duplicate n -gram repetition in natural language prose. These findings show that functional decomposition offers a principled, vocabulary-efficient foundation for language modeling, motivating further exploration across broader scripts, morphological framework extensions, and production-scale architectures.

A Diacritic Operators Specification

Table 5 lists the 13 dedicated 1-parameter combining diacritic operators supported in the current implementation, mapped to the U+E101–U+E10D Unicode Private Use Area plane.

Table 5: Combining Diacritic Operators Specification

Operator / Opcode	Params	Action
TILDE (U+E101)	pos	Apply combining tilde (\u{0303}) to the character at pos.
ACUTE (U+E102)	pos	Apply combining acute (\u{0301}) to the character at pos.
GRAVE (U+E103)	pos	Apply combining grave (\u{0300}) to the character at pos.
CIRCUMFLEX (U+E104)	pos	Apply combining circumflex (\u{0302}) to the character at pos.
DIAERESIS (U+E105)	pos	Apply combining diaeresis (\u{0308}) to the character at pos.
MACRON (U+E106)	pos	Apply combining macron (\u{0304}) to the character at pos.
DOT_ABOVE (U+E107)	pos	Apply combining dot above (\u{0307}) to the character at pos.
RING_ABOVE (U+E108)	pos	Apply combining ring above (\u{030A}) to the character at pos.
DOUBLE_ACUTE (U+E109)	pos	Apply combining double acute (\u{030B}) to the character at pos.
CARON (U+E10A)	pos	Apply combining caron (\u{030C}) to the character at pos.
COMMA_BELOW (U+E10B)	pos	Apply combining comma below (\u{0326}) to the character at pos.
CEDILLA (U+E10C)	pos	Apply combining cedilla (\u{0327}) to the character at pos.
OGONEK (U+E10D)	pos	Apply combining ogonek (\u{0328}) to the character at pos.

B Worked Encoding Examples

Table 6: Worked Encoding Examples

Input	Operators	Base	Encoded prefix
Hello	CAPITALIZE(0)	hello	\u{E100}\u{E000}
Héllö	ACUTE(1) CAPITALIZE(0)	hello	\u{E102}\u{E001}\u{E100}\u{E000}
#####	REPEAT(0, 6)	#	\u{E200}\u{E000}\u{E006}
(4 spaces)	REPEAT(0, 4)	(space)	\u{E200}\u{E000}\u{E004}
abcbabcabc	MULTIREPEAT(0, 3, 3)	abc	\u{E201}\u{E000}\u{E003}\u{E003}

Data and Code Availability

All tokenizer configurations, training scripts, evaluation pipelines, dataset acquisition scripts, and result files are publicly available at <https://github.com/connor-makowski/functionalizer>. A working implementation is available at <https://github.com/connor-makowski/tokenizers> on the functionalizer branch. Select trained Funcitonalizer models are available at <https://hf.co/collections/mrkwanzaa/functionalizer-100m>.

References

- Asgari, E., El Kheir, Y., & Sadraei Javaheri, M. A. (2025). MorphBPE: A Morpho-Aware Tokenizer Bridging Linguistic Complexity for Efficient LLM Training Across Morphologies. *arXiv preprint arXiv:2502.00894*. <https://arxiv.org/abs/2502.00894>
- Bayram, M. A., Fincan, A. A., Gümüş, A. S., Karakaş, S., Diri, B., Yıldırım, S., & Çelik, D. (2025). Tokens with Meaning: A Hybrid Tokenization Approach for Turkish. *arXiv preprint arXiv:2508.14292*. <https://arxiv.org/abs/2508.14292>

- Bérard, A., Calapodescu, I., & Roux, C. (2019). Naver Labs Europe’s Systems for the WMT19 Machine Translation Robustness Task. In *Proceedings of the Fourth Conference on Machine Translation (Volume 2: Shared Task Papers, Day 1)* (pp. 526–532). Association for Computational Linguistics. <https://aclanthology.org/W19-5361/>
- CodeParrot Team and Hugging Face (2022). *GitHub-code dataset*. <https://huggingface.co/datasets/codeparrot/github-code>
- Creutz, M., & Lagus, K. (2002). Unsupervised Discovery of Morphemes. In *Proceedings of the ACL-02 Workshop on Morphological and Phonological Learning* (pp. 21–30). Association for Computational Linguistics. <https://aclanthology.org/W02-0603/>
- Creutz, M., & Lagus, K. (2007). Unsupervised Models for Morpheme Segmentation and Morphology Learning. *ACM Transactions on Speech and Language Processing*, 4(1), Article 3. <https://doi.org/10.1145/1187415.1187418>
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., and others (2024). The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*. <https://arxiv.org/abs/2407.21783>
- Etchegoyhen, T., & Gete, H. (2020). To Case or not to case: Evaluating Casing Methods for Neural Machine Translation. In *Proceedings of the Twelfth Language Resources and Evaluation Conference* (pp. 3752–3760). European Language Resources Association. <https://aclanthology.org/2020.lrec-1.463/>
- Flytech (2023). *Python-codes-25k dataset*. <https://huggingface.co/datasets/flytech/python-codes-25k>
- Forsythe, A. (2023). *TokenMonster: Ungreedy Subword Tokenizer and Vocabulary Trainer for Python, Go, C++ & Javascript* [Computer software]. <https://github.com/alsadairforsythe/tokenmonster>
- Gorman, K., & Pinter, Y. (2025). Don’t Touch My Diacritics. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)* (pp. 285–291). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.naacl-short.25>
- Jain, R., Khayrallah, H., Grundkiewicz, R., & Junczys-Dowmunt, M. (2023). Perplexity-Driven Case Encoding Needs Augmentation for CAPITALIZATION Robustness. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)* (pp. 146–156). Association for Computational Linguistics. <https://aclanthology.org/2023.ijcnlp-short.17/>
- Kallini, J., Murty, S., Manning, C. D., Potts, C., & Csordás, R. (2025). MrT5: Dynamic Token Merging for Efficient Byte-level Language Models. In *The Thirteenth International Conference on Learning Representations (ICLR)*. arXiv:2410.20771. <https://openreview.net/forum?id=VYWBmq1L7H>
- Kudo, T. (2018). Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 66–75). Association for Computational Linguistics. <https://aclanthology.org/P18-1007/>
- Land, S., & Arnett, C. (2025). BPE Stays on SCRIPT: Structured Encoding for Robust Multilingual Pretokenization. In *Tokenization Workshop (TokShop) at ICML 2025*. <https://arxiv.org/abs/2505.24689>
- Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016). Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*. <https://arxiv.org/abs/1609.07843>
- Penedo, G., Kydlíček, H., Allal, L. B., Lozhkov, A., Mitchell, M., Raffel, C., von Werra, L., & Wolf, T. (2024). The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*. <https://arxiv.org/abs/2406.17557>

- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report*. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- Rexline, S. J., & Robert, L. (2011). Substitution Coder — A Reversible Data Transform for Lossless Text Compression. In *2011 8th International Conference on Information, Communications & Signal Processing (ICICS)* (pp. 1–5). IEEE. <https://doi.org/10.1109/ICICS.2011.6173125>
- Samuel, D., & Øvrelid, L. (2023). Tokenization with Factorized Subword Encoding. In *Findings of the Association for Computational Linguistics: ACL 2023* (pp. 14143–14161). Association for Computational Linguistics. <https://aclanthology.org/2023.findings-acl.890/>
- Schuster, M., & Nakajima, K. (2012). Japanese and Korean Voice Search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 5149–5152). IEEE. <https://doi.org/10.1109/ICASSP.2012.6289079>
- Semenov, K., & Popel, M. (2025). InCa and InDia: Inline Casing and Diacritization Preprocessing For Robust-to-Noise Tokenization and Interpretability. In *Tokenization Workshop (TokShop) at ICML 2025*. <https://openreview.net/forum?id=9GwVWxjVmN>
- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1715–1725). Association for Computational Linguistics. <https://aclanthology.org/P16-1162/>
- Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., & Raffel, C. (2022). ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models. *Transactions of the Association for Computational Linguistics*, 10, 291–306. https://doi.org/10.1162/tac1_a_00461
- Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., & Lewis, M. (2023). MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)* (pp. 78808–78823). <https://arxiv.org/abs/2305.07185>